ASP.NET Core・MVC 入門

MVC アプリを作成し、画面表示の流れを理解する

対応環境　Visual Studio 2022（17.8 以降）／.NET 8.0

Version 1.0

学習内容

MvcBasicSample を新規作成し、文字列の応答から View による画面表示までを実装する。URL から処理が選ばれ、HTML がブラウザへ返る流れを確認する。

実習の前提

Windows 版 Visual Studio 2022 のバージョン 17.8 以降と.NET 8.0 を使用する。C#の変数、クラス、メソッド、継承の基本を前提とする。Web 特有の用語は本文で説明する。

【重要】実習のプロジェクト名は MvcBasicSample に統一する。コード中の名前空間も同じ名前を使う。

構成

STEP 1　プロジェクトを新規作成する

STEP 2　テンプレートを起動する

Web と MVC の仕組み

STEP 3　HelloController を新規作成する

Routing と URL の対応

STEP 4　View を新規作成して表示する

STEP 5　Program.cs を確認する

到達確認と問題の切り分け

本版の内容

Version 1.0　Visual Studio 2022・.NET 8.0 対応、作成手順、Routing、MVC 関係図、ViewData の初出説明、実行確認を整備

STEP 1 プロジェクトを新規作成する

対象ファイル

作成後の MvcBasicSample.csproj とプロジェクト一式

新規作成か既存編集か

新規作成

操作手順

1　Visual Studio Installer で Visual Studio 2022 の［変更］を開き、［ASP.NET と Web 開発］が導入済みか確認する。.NET 8.0 SDK がない場合は追加する。

2　Visual Studio 2022 を起動し、［新しいプロジェクトの作成］を選ぶ。既に起動中なら［ファイル］→［新規作成］→［プロジェクト］を開く。

3　検索欄に「ASP.NET Core」と入力する。言語を C#に絞り、［ASP.NET Core Web アプリ（Model-View-Controller）］を選んで［次へ］を押す。

4　プロジェクト名とソリューション名を MvcBasicSample にする。保存場所を確認して［次へ］を押す。ソリューションはプロジェクトをまとめる単位である。

5　追加情報でフレームワークを［.NET 8.0（長期的なサポート）］、認証の種類を［なし］にする。［HTTPS 用の構成］は有効にする。

6　［コンテナーサポート］は無効にする。［最上位レベルのステートメントを使用しない］はチェックしない。ほかの連携機能は追加せず［作成］を押す。

完成コード

この STEP ではコードを入力しない。［作成］を押すと、選択したテンプレートをもとに Program.cs と各フォルダーが自動生成される。Program.cs の完成コードは STEP 5 で確認する。

コードの意味

.NET は C#の開発と実行の基盤、ASP.NET Core はその上で Web アプリを作るための仕組みである。テンプレートは動作する最小構成を生成する。

実行確認

ソリューション エクスプローラーに MvcBasicSample があり、Controllers、Models、Views、Program.cs を確認できればよい。表示されない場合は［表示］→［ソリューション エクスプローラー］を開く。

【注意】MVC を選ぶ。名前が似た Razor Pages、Web API、.NET Framework のテンプレートでは本書の手順と一致しない。

STEP 2 テンプレートを起動する

対象ファイル

Program.cs、Properties/launchSettings.json、生成済みフォルダ

新規作成か既存編集か

既存ファイルの確認のみ

操作手順

1　実行ボタン横の起動プロファイルで https を選ぶ。本書ではプロジェクト自身を起動するプロファイルを使用する。

2　F5 で起動する。自分の開発用 localhost 証明書を信頼する確認が出た場合は内容を確認して許可する。

3　Home 画面が表示されたら URL を控える。以降は同じ https://localhost:ポート番号を使い、末尾のパスだけを変更する。

4　編集前に Shift＋F5 で停止する。編集後は保存、［ビルド］→［ソリューションのビルド］、F5 の順で確認する。

完成コード

コード変更はない。launchSettings.json は実行時の URL などを保持する設定ファイルであり、この段階では書き換えない。

コードの意味

F5 はデバッグ付き実行、Ctrl＋F5 はデバッグなし実行である。どちらでも画面を表示できるが、本書は停止操作が揃う F5 を基本とする。

実行確認

Home 画面が表示され、ビルドのエラーが 0 件であれば成功。ポート番号は環境ごとに異なり、教材の文字列「ポート番号」をそのまま入力しない。

作成直後のフォルダーとファイルの役割

場所	役割
Controllers	ブラウザからの要求を受け取る Controller クラスを置くフォルダー
Models	アプリで扱うデータを表す Model クラスを置くフォルダー
Views	画面となる.cshtml ファイルを置くフォルダー
wwwroot	CSS、JavaScript、画像などの静的ファイルを置くフォルダー
Program.cs	アプリの起動に必要な機能や URL の対応規則を設定するファイル
Properties/launchSettings.json	Visual Studio から実行するときの URL や起動方法を設定するファイル

【補足】「新しいネイティブ TypeScript コンパイラを使用しています」という案内はエラーではない。テンプレートに JavaScript が含まれるため表示されることがある。通知を閉じてもよく、機能を無効にする必要はない。TypeScript は JavaScript に型などを加えた言語で、本書では記述しない。

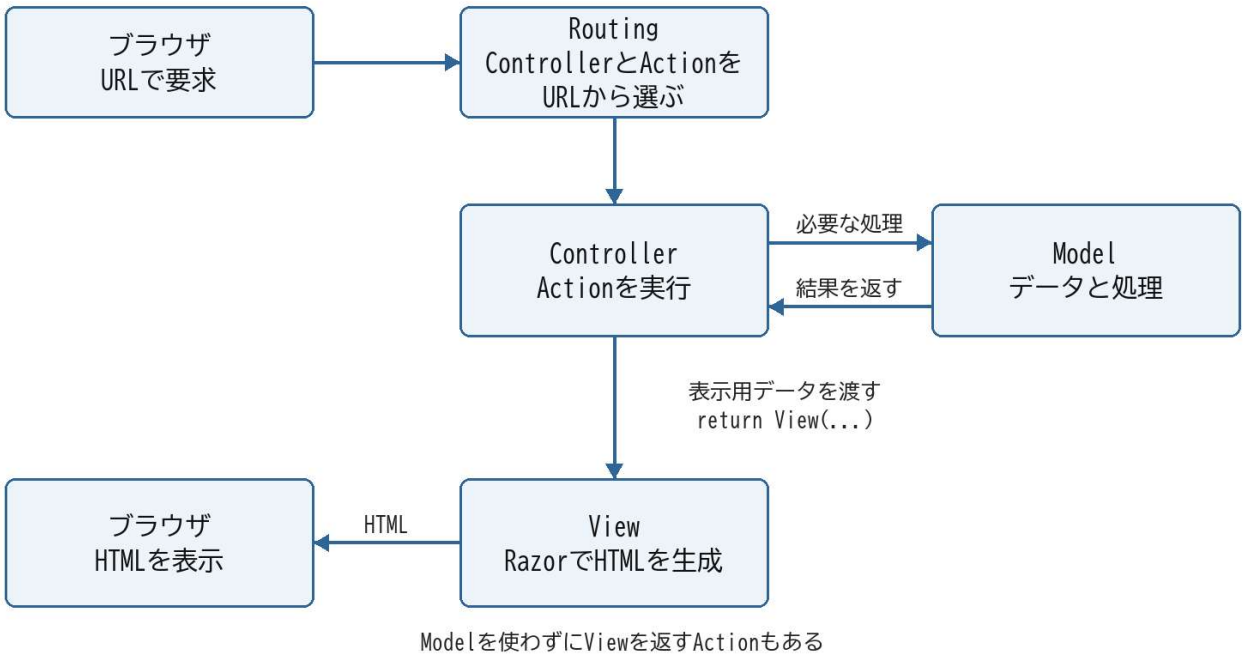
Web と MVC の仕組み

ブラウザは URL を使って画面を要求する側であり、ASP.NET Core アプリはその要求を受け取って応答を返す側である。HTTP は要求と応答をやり取りするための通信規則で、HTTPS はその通信を暗号化する。

localhost は自分の PC を指す名前、ポート番号はその PC 内の接続先を区別する番号である。Kestrel は ASP.NET Core に組み込まれた Web サーバーで、本書の起動プロファイルでは HTTP／HTTPS の要求を受け取る。

役割	担当
Model	Controller や View で扱うデータを表す
View	Controller から受け取ったデータを使って HTML を生成する
Controller	URL に対応する Action で要求を受け取り、返す内容を決める

MVC の関係図



ブラウザから届いた要求は Kestrel が受け取り、ASP.NET Core へ渡す。Routing は URL を確認して実行する Controller と Action を選ぶ。Action は Controller 内にある public メソッドで、必要に応じて Model を使い、View へ表示用のデータを渡す。

【重要】この関係図は MVC の説明と実装手順で共通に用いる。今回の Hello 画面は Model をまだ使わず、Controller から直接 View を返す最小例である。

GET は主に画面やデータの取得、POST は主に登録や変更のためのデータ送信に使う。本書では URL を開いたときの GET 要求を扱う。

STEP 3 HelloController を新規作成する

対象ファイル

Controllers/HelloController.cs

新規作成か既存編集か

新規作成

操作手順

1　停止状態で Controllers フォルダを右クリックし、［追加］→［クラス］を選ぶ。

2　ファイル名を HelloController.cs にして作成する。生成された内容をすべて下のコードに置き換えて保存する。

完成コード

```
using Microsoft.AspNetCore.Mvc;  // Controller と IActionResult を使用

namespace MvcBasicSample.Controllers;

// URL の Hello に対応する要求を受け取る Controller
public class HelloController : Controller
{
    // /Hello/Index で呼び出される Action
    public IActionResult Index()
    {
        // View を使用せず文字列を HTTP の応答として返す
        return Content("はじめての ASP.NET Core");
    }
}
```

コードの意味

using は別の名前空間にある型を利用するための指定である。namespace はクラスの所属名を表し、フォルダと同じ構成に揃える。

HelloController は Controller を継承する。public の Index メソッドが Action になり、IActionResult は文字列や画面などの応答結果を表す型である。

Content(...)は、指定した文字列を HTTP の応答として返す。この時点では View を探さない。

実行確認

ビルド後に F5 で起動する。アドレスバーのパスを/Hello/Index に置き換える。

「はじめての ASP.NET Core」が表示されれば成功。Home のメニューや装飾が表示されないのは文字列だけを返しているためである。

【補足】クラスを作る操作はコードを手入力するための準備である。Controller と View をまとめて自動生成する機能はこの実習では使わない。

Routing と URL の対応

Routing は、ブラウザから送られた URL を Controller の Action へ対応付ける仕組みである。MVC では Controller の Action が最初に要求を受け取り、必要なデータを準備して、文字列を返すか View を使うかを決める。そのため、/Hello/Index を開いたときは View ではなく HelloController の Index()を先に探す。

【重要】Views フォルダは公開用のファイル置き場ではない。/Hello/Index を開いても、最初から Views/Hello/Index.cshtml を探すのではない。Action が return View()を実行して初めて View を選ぶ。

Program.cs にある標準の対応規則

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

これは既存の設定の抜粋であり、ここでは追記しない。{controller}と{action}に URL の各部分が入り、省略時は Home と Index を使う。{id?}は省略可能な追加の値で、本書では使わない。

URL の部分	対応
Hello	HelloController の Controller を除いた名前
Index	HelloController 内の Index()メソッド
/Hello/Index	HelloController の Index()を実行
/Hello	Action を省略して Index()を実行
/	HomeController の Index()を実行

文字列を返す場合と View を使う場合

```
/Hello/Index
    ↓ Routing
HelloController.Index()
    ├── return Content(...) → 文字列をブラウザへ返す
    └── return View()       → View を探して HTML を生成
                               ↓
                      Views/Hello/Index.cshtml
```

引数なしの View()は実行中の Action 名を使う。通常は Views/Hello/Index.cshtml を探し、見つからなければ Views/Shared/Index.cshtml も候補になる。本書では Hello フォルダ内に作成する。

【注意】/HelloController/Index や/Views/Hello/Index.cshtml は、この MVC ルートで目的の Action を指定する URL ではない。

STEP 4 View を新規作成して表示する

対象ファイル

Controllers/HelloController.cs／Views/Hello/Index.cshtml

新規作成か既存編集か

Controller は既存編集、Hello フォルダと View は新規作成

操作手順

1　HelloController.cs の内容を下の完成コードへ置き換える。

2　Views を右クリックし、［追加］→［新しいフォルダー］で Hello を作る。

3　Hello を右クリックし、［追加］→［新しい項目］で「Razor ビュー - 空」などの空の MVC View を選び、Index.cshtml を作る。見つからない場合はテキストファイルを作成して拡張子を.cshtml にする。

4　View の内容も下のコードに置き換える。@page は付けない。2 ファイルを保存してから実行する。

完成コード

Controllers/HelloController.cs

```
using Microsoft.AspNetCore.Mvc;  // Controller と IActionResult を使用

namespace MvcBasicSample.Controllers;

// URL の Hello に対応する要求を受け取る Controller
public class HelloController : Controller
{
    // /Hello/Index で呼び出される Action
    public IActionResult Index()
    {
        // Views/Hello/Index.cshtml を使って HTML を生成する
        return View();
    }
}
```

【補足】ViewData は、1 回の画面表示で使用する値を、文字列のキー名で保存・取得する仕組みである。次のコードでは Title というキーに Hello を保存する。共通レイアウトは同じキーから値を取り出し、ブラウザのタブに表示するタイトルを作る。

Views/Hello/Index.cshtml

```
@{
    ViewData["Title"] = "Hello";  // タブ用のタイトル
}

<h1>はじめての MVC</h1>  @* 画面内の見出し *@
<p>Controller から View が表示されました</p>
```

STEP 4 コードの意味と実行確認

コードの意味

return View()は Index という View で HTML を生成する応答を返す。Razor は.cshtml 内の C#をサーバー側で実行し、結果を HTML へ組み込む仕組みである。

@{ ... }は C#の処理を書く範囲である。<h1>は HTML の見出し、<p>は段落を表す。@* ... *@は Razor のコメントで、ブラウザへ送る HTML には含まれない。

ViewData["Title"]の Title は情報を取り出すための名前であり、予約された変数名ではない。MVC テンプレートでは共通レイアウトがこのキーを使う約束になっている。

ViewData の値は今回の要求の画面生成で使う。別の URL へ移動したときに、その値が自動的に引き継がれるわけではない。

実行確認

ビルドして F5 で起動し、/Hello/Index を開く。「はじめての MVC」と「Controller から View が表示されました」が表示されれば成功。

タブのタイトルは「Hello - MvcBasicSample」になる。画面内の見出しとタブのタイトルは別のものである。

ViewData の値がタブに届く流れ

```
Views/Hello/Index.cshtml
    ViewData["Title"] = "Hello";
                  ↓
Views/Shared/_Layout.cshtml
    <title>@ViewData["Title"] - MvcBasicSample</title>
                  ↓
ブラウザへ届く HTML
    <title>Hello - MvcBasicSample</title>
```

上の図は仕組みを説明する抜粋であり、_Layout.cshtml を置き換えるコードではない。レイアウトは各画面の共通の外枠で、テンプレートが既に用意している。

【注意】C#の文字列には半角の"を使う。Word などの自動変換で“ ”へ変わった場合は、Visual Studio 上で半角へ直す。

STEP 5　Program.cs を確認する

対象ファイル

プロジェクト直下の Program.cs

新規作成か既存編集か

既存編集

操作手順

1　アプリを停止し、Program.cs を開く。.NET 8.0 の構成に揃えるため、ファイル全体を下のコードに置き換えて保存する。

完成コード

```
var builder = WebApplication.CreateBuilder(args);

// Controller と View を使う MVC の機能を登録する
builder.Services.AddControllersWithViews();

var app = builder.Build();  // 登録した設定から実行用のアプリを作成

// 開発環境以外で使うエラー画面と HTTPS の設定
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();  // HTTP から HTTPS へ転送
app.UseStaticFiles();       // wwwroot にある CSS や JavaScript などを配信
app.UseRouting();           // URL と Controller の Action を対応付ける
app.UseAuthorization();     // 設定されているアクセス許可を確認

// URL を Controller と Action に対応付ける
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();                  // Web サーバーを起動して要求の受信を開始
```

コードの意味

CreateBuilder は設定ファイルや起動時の引数を読み込む準備を行う。AddControllersWithViews は Controller と View を使用する MVC の機能を登録する。Build は登録済みの設定から実行用のアプリを作り、Run は Web サーバーを起動して要求の受信を開始する。

Use で始まる処理は、ブラウザから届いた要求を記述順に処理する部品である。こうした部品を Middleware（ミドルウェア）と呼ぶ。

実行確認

ビルドのエラーが 0 件で、/と/Hello/Index の両方を開けることを確認する。

Program.cs の補足と到達確認

起動時の設定を読む

IsDevelopment は開発環境かどうかを調べる。開発環境以外では UseExceptionHandler で既存の/Home/Error へエラー処理を渡し、UseHsts でブラウザに HTTPS を使うよう通知する設定を有効にする。

UseHttpsRedirection は HTTP から HTTPS へ転送する。UseAuthorization はアクセス許可を確認する仕組みであり、このコードだけでログイン機能が作られるわけではない。

UseStaticFiles は wwwroot にある CSS、JavaScript、画像などの静的ファイルをブラウザへ配信する。静的ファイルは、C#と Razor で HTML を生成する View とは異なる。

【補足】本書は.NET 8.0 の MVC テンプレートに合わせて UseStaticFiles を使用する。wwwroot に置いたファイルを配信するため、UseRouting より前に記述する。

問題を切り分ける

症状	最初に確認する場所
404 になる	/Hello/Index という URL と HelloController の Index()
View が見つからない	Views/Hello/Index.cshtml の場所と名前
接続できない	起動状態と実際の https プロファイルのポート番号
古い画面が出る	保存後に停止して再ビルドし、再起動したか
赤い下線がある	エラー一覧の「ビルド」の内容を確認する

到達確認

- ☐ Visual Studio 2022 と.NET 8.0 で MvcBasicSample を作成できる
- ☐ localhost、ポート、Kestrel の役割を説明できる
- ☐ URL から Controller の Action を選ぶ理由を説明できる
- ☐ Content()と View()で返す内容の違いを説明できる
- ☐ ViewData のキーと値、画面見出しとタブタイトルを区別できる
- ☐ MVC の関係図を使い HTML が返るまでの流れを説明できる

参照資料

Microsoft Learn　ASP.NET Core MVC の概要と開始手順

https://learn.microsoft.com/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-8.0

Microsoft Learn　ASP.NET Core の静的ファイル

https://learn.microsoft.com/aspnet/core/fundamentals/static-files?view=aspnetcore-8.0